1.Objective Add P.L(presentation logic) Validations supplied in back end

Steps
1.1 We Have already added , validation dependency starter in Spring boot project.
(confirm from pom.xml)

1.2 Identify validation rules , add the annotations on POJO(Entity/DTO) properties.
eg : @NotBlank, @Pattern, @Min, @Max,@NotNull,@Past,@Future.....
Imported from javax.validation , org.hibernate.validator
(refer to templates.txt under)
eg : first name : can't be blank .(min : 4 chars , max =20 chars)
last name : can't be blank
valid email
valid strong password (alpha numeric, special character , min 5 max 20)
eg : ((?=.*\\d)(?=.*[a-z])(?=.*[#@$*]).{5,20})
reg amount in the range : 500 ---5000
reg date must be in future
Test it with postman client
(Same annotations are used in Spring MVC standalone App , in P.L: presentation logic validations)
eg : @NotBlank , @NotNull,@Email, @Pattern,@Future ,@Range,@Min,@Max...

1.3 For validating RequestBody : add @Valid annotation in addition to @RequestBody
SC performs un marshalling + validation

Observation : HTTP status 400 , BUT entire error stack trace sent to clnt.
Exception : MethodArgumentNotValidException

Is it a desirable ?? NO

1.4 For Path Variables/request params : @Validated , at class level on controller class + validation annotations (eg : @NotNull, @Email...) on the method argument
Exception raised : ConstraintViolationException

2. Any problems observed on the client side ?

YES : Since spring boot supplies a default exception handler , entire stack trace , along with exception details are sent to the front end.

NOTE :
1. Validation failures CAN NOT be caught by rest controller method level exc handling using try-catch .
It's automatically handled by Spring Boot supplied default exception handler.
Problem :Complete Error stack trace is sent to the clnt.

2. B.L failures (eg : resource not found , dup email ...) CAN BE caught by controller method level exc handling(try-catch) --> but resulting into repeatative exc handling
(i.e multiple try-catch)

Instead :
How to avoid both of these problems ?
Solution : Add centralized (global) exception handler

Why ?

A good REST API should handle the exceptions properly and send the proper response to the user. The user(REST Client) should not be rendered with any unhandled exception. A REST API developer will have two requirements related to error handling.

1. Centralized place(class) for Error handling
2. Similar Error Response body with a proper HTTP status code across all API endpoints

Steps :
1. Create a separate class annotated with
Class level annotation :
@RestControllerAdvice is the combination of both @ControllerAdvice at the class level and @ResponseBody added implicitly over all exc handling methods.

The @ControllerAdvice annotation was first introduced in Spring 3.2.

We can use the @ControllerAdvice annotation for handling exceptions in the RESTful Services but we need to add @ResponseBody separately.

The differences between @RestControllerAdvice and @ControllerAdvice is :

@RestControllerAdvice = @ControllerAdvice + @ResponseBody. - we can use it in REST web services.

@ControllerAdvice - We can use in both MVC and Rest web services, BUT you need to provide the @ResponseBody if we use this in Restful web services.

Steps
1. Create a separate class annotated with @RestControllerAdvice
To tell SC , following class is a common advice to : rest controllers --regarding exc handling (cross cutting concern=common task)

3. For validation failures , triggered by @Valid annotation
Exception to handle : MethodArgumentNotValidException
Extract map of Field Errors --send it to the caller(front end) by wrapping it in the RespEntity.
Exc class : MethodArgNotValidException
It's super class : BindException has :
Method --List<FieldError> getFieldErrors() .
FieldError class : getField(): affected field name , getDefaultMessage() : error mesg

eg : @ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<?>
handleMethodArgumentNotValidException(MethodArgumentNotValidException e)
{....}
o.s.http.ResponseEntity : a generic class , that represents ENTIRE response packet (containing SC code | header | resp body)

4. For ResourceNotFoundException or similar B.L exceptions :
Add method in global exc handler annotated with
@ExceptionHandler

Instead of sending err mesg as a plain string , wrap it in Error response object n send it to the front end

for simpler processing
Use DTO pattern : Data Transfer object
ApiResponse : message , timestamp

Links :
https://www.baeldung.com/java-bean-validation-not-null-empty-blank